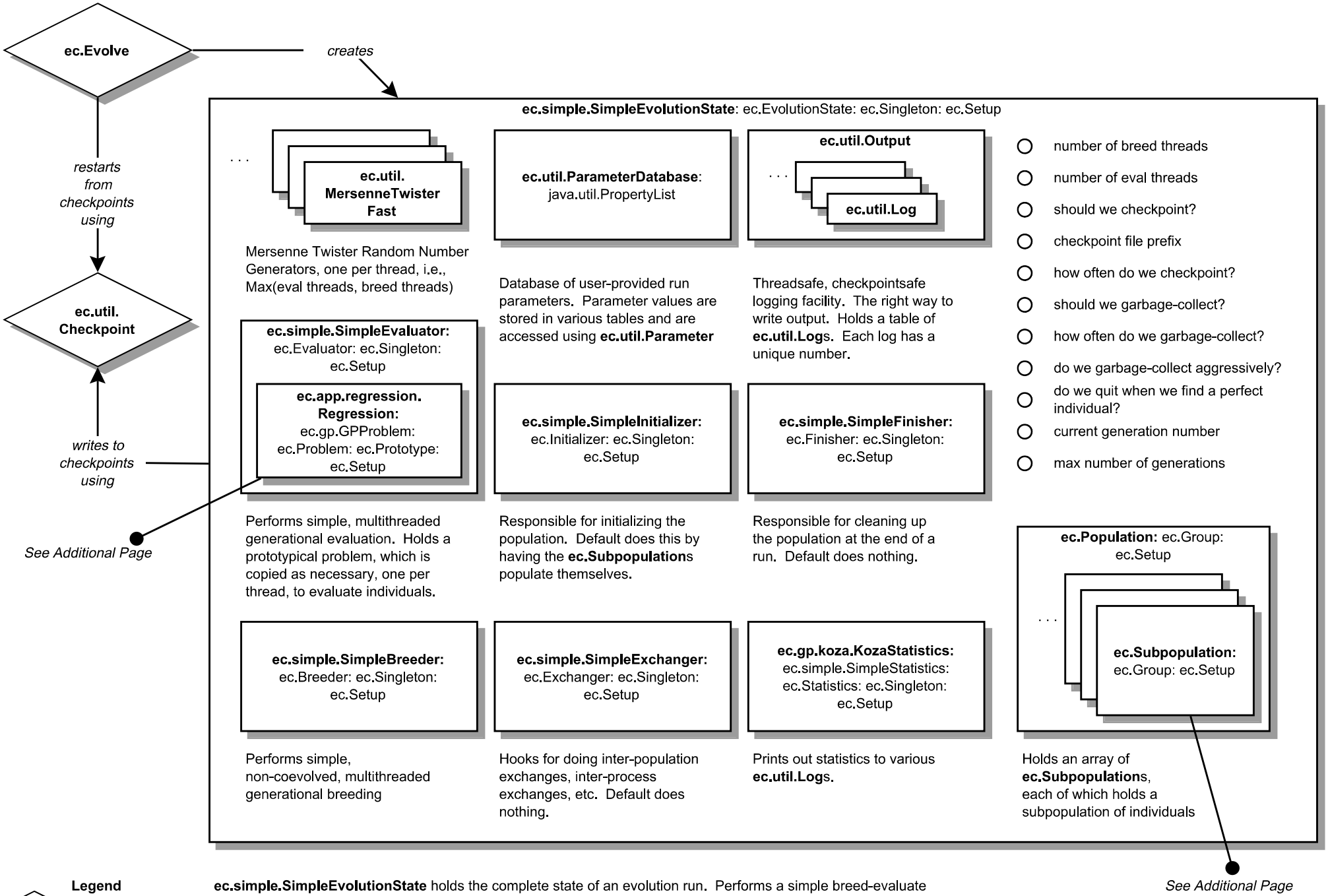
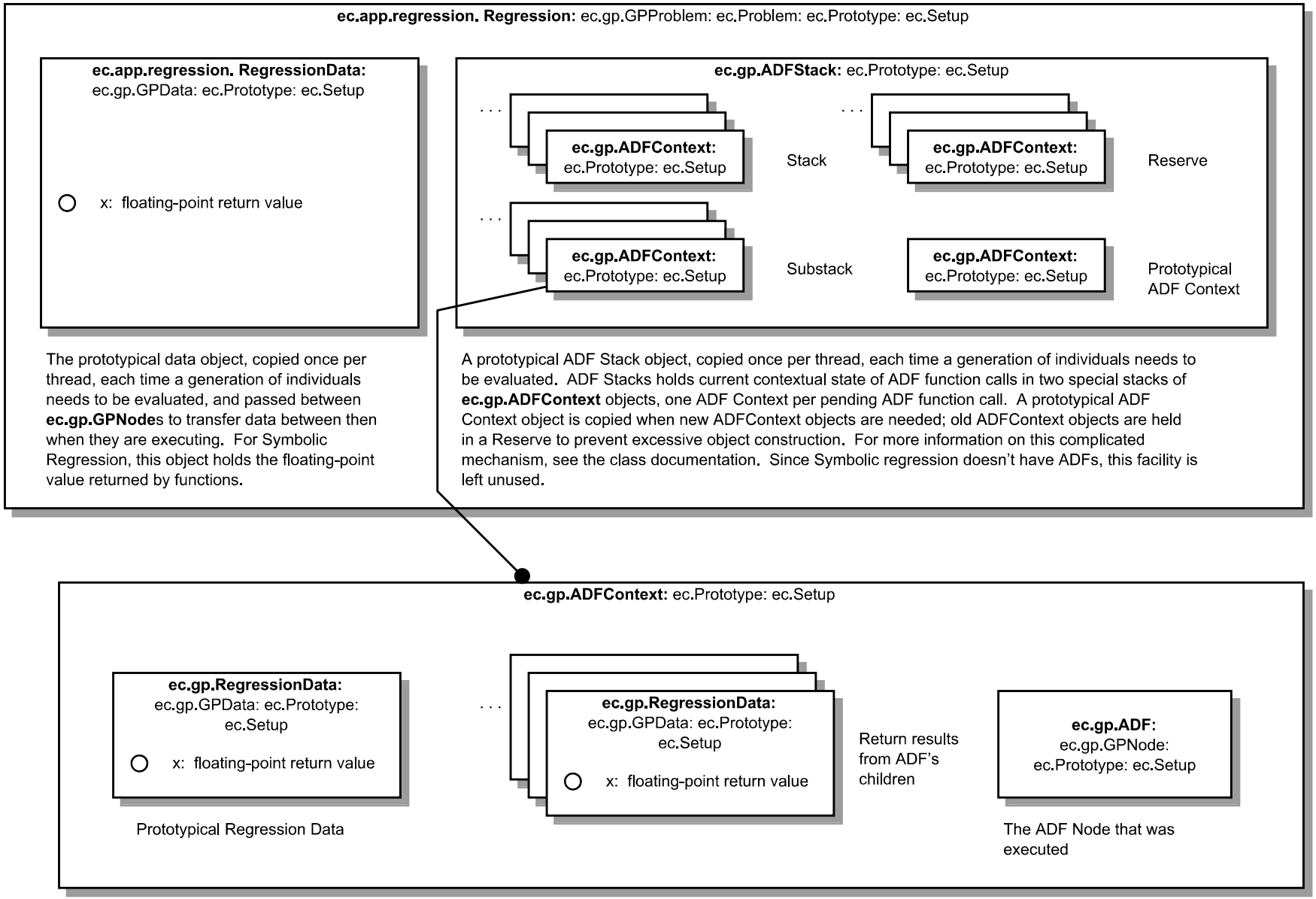


ec.simple.SimpleEvolutionState holds the complete state of an evolution run. Performs a simple breed-evaluate loop on individuals in its population. Every object stored within this object is serializable, and so can be written out to a checkpoint file using **ec.util.Checkpoint.** The evolution state is also passed around to among objects a lot so they can access its random number generators, parameter database, and logs. This is done, of course, in a threadsafe manner.



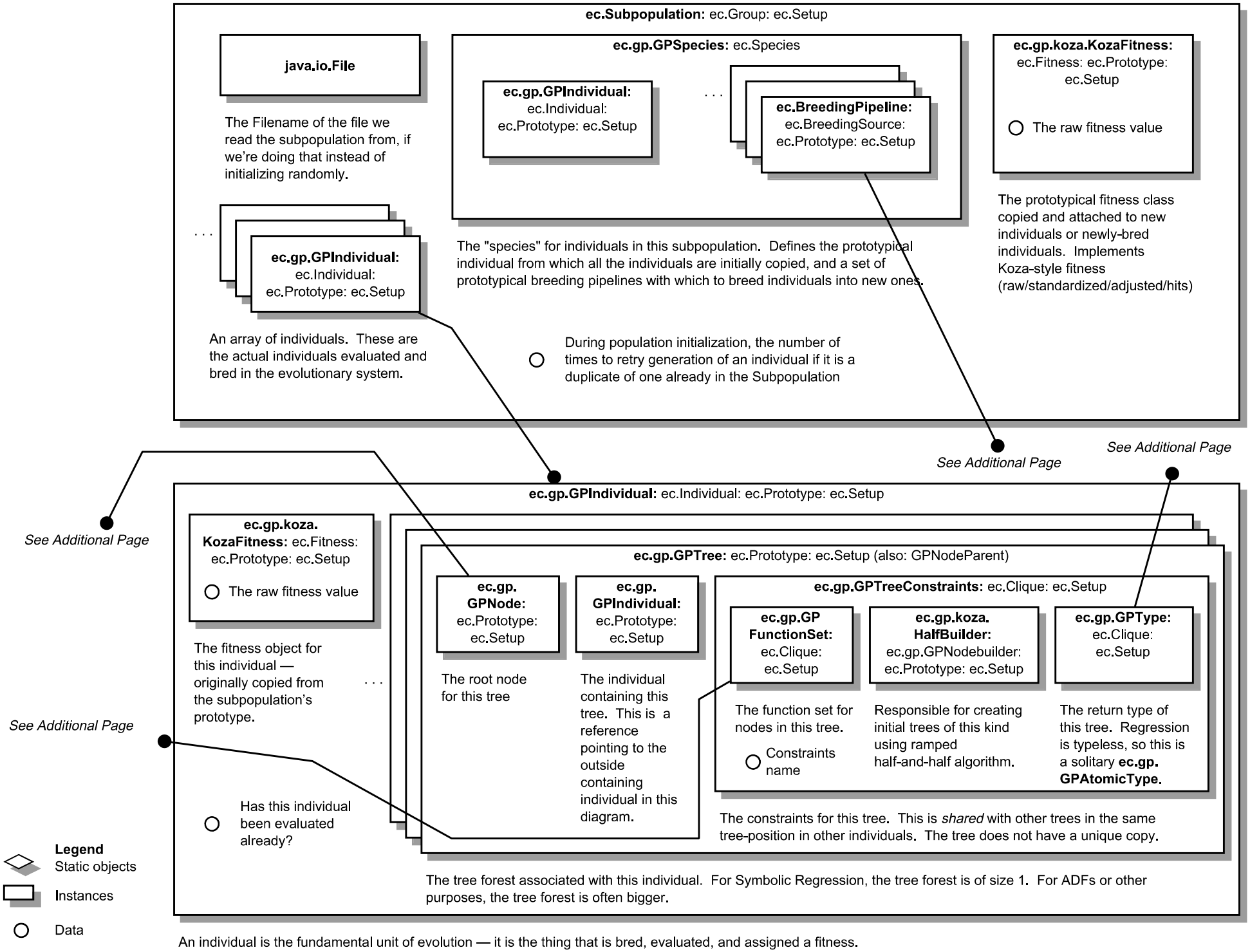
Legend

Static objects

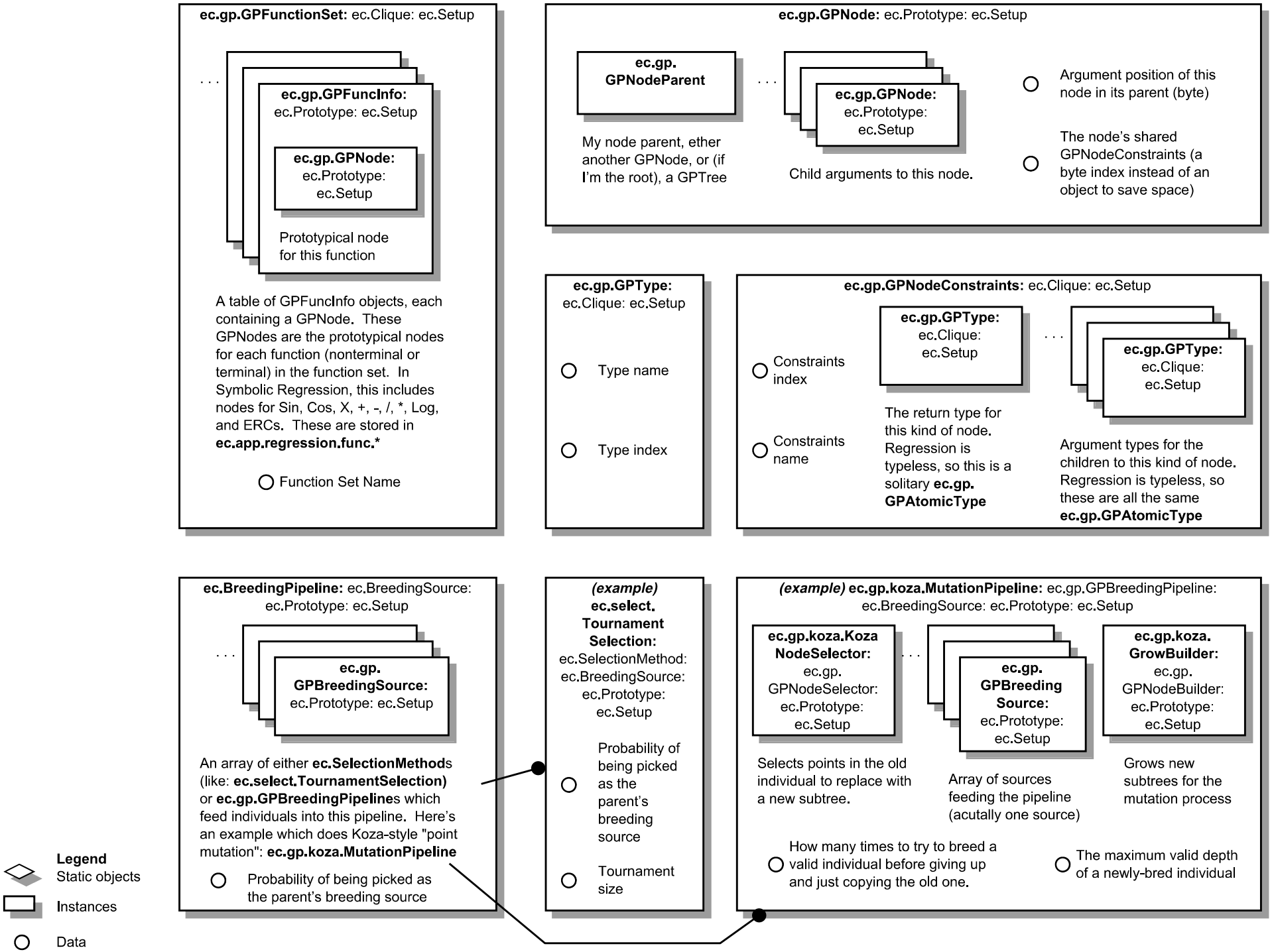
Instances

Data

An ADF Context holds the current execution context of an ADF function call. The ADF ("Automatically Defined Function", see Koza-I and Koza-II) called is stored in the context, as is the returned data from its children nodes. A prototypical data object is also held from which new children results are created if necessary. This data object is typically the same kind of prototype as the one used in the problem, in this case, **ec.app.regression.Regression.** ADF Contexts are also used for ADMs ("Automatically Defined Macros"). Since Symbolic Regression doesn't have ADFs, this facility is left unused.



An individual is the fundamental unit of evolution — it is the thing that is bred, evaluated, and assigned a fitness.


ec.gp.GPFunctionSet: ec.Clique: ec.Setup
. . .
ec.gp.GPFuncInfo: ec.Prototype: ec.Setup
ec.gp.GPNode: ec.Prototype: ec.Setup
Prototypical node for this function
A table of GPFuncInfo objects, each containing a GPNode. These GPNodes are the prototypical nodes for each function (nonterminal or terminal) in the function set. In Symbolic Regression, this includes nodes for Sin, Cos, X, +, -, /, *, Log, and ERCs. These are stored in ec.app.regression.func.*
Function Set Name
ec.gp.GPNode: ec.Prototype: ec.Setup
ec.gp. GPNodeParent
My node parent, ether another GPNode, or (if I'm the root), a GPTree
. . .
ec.gp.GPNode: ec.Prototype: ec.Setup
Child arguments to this node.
Argument position of this node in its parent (byte)
The node's shared GPNodeConstraints (a byte index instead of an object to save space)
ec.gp.GPType: ec.Clique: ec.Setup
Type name
Type index
ec.gp.GPNodeConstraints: ec.Clique: ec.Setup
Constraints index
Constraints name
ec.gp.GPType: ec.Clique: ec.Setup
The return type for this kind of node. Regression is typeless, so this is a solitary ec.gp. GPAtomicType
. . .
ec.gp.GPType: ec.Clique: ec.Setup
Argument types for the children to this kind of node. Regression is typeless, so these are all the same ec.gp.GPAtomicType
ec.BreedingPipeline: ec.BreedingSource: ec.Prototype: ec.Setup
. . .
ec.gp. GPBreedingSource: ec.Prototype: ec.Setup
An array of either ec.SelectionMethods (like: ec.select.TournamentSelection) or ec.gp.GPBreedingPipelines which feed individuals into this pipeline. Here's an example which does Koza-style "point mutation": ec.gp.koza.MutationPipeline
Probability of being picked as the parent's breeding source
(example) ec.select. Tournament Selection: ec.SelectionMethod: ec.BreedingSource: ec.Prototype: ec.Setup
Probability of being picked as the parent's breeding source
Tournament size
(example) ec.gp.koza.MutationPipeline: ec.gp.GPBreedingPipeline: ec.BreedingSource: ec.Prototype: ec.Setup
ec.gp.koza.Koza NodeSelector: ec.gp. GPNodeSelector: ec.Prototype: ec.Setup
Selects points in the old individual to replace with a new subtree.
. . .
ec.gp. GPBreeding Source: ec.Prototype: ec.Setup
Array of sources feeding the pipeline (acutally one source)
ec.gp.koza. GrowBuilder: ec.gp. GPNodeBuilder: ec.Prototype: ec.Setup
Grows new subtrees for the mutation process
How many times to try to breed a valid individual before giving up and just copying the old one.
The maximum valid depth of a newly-bred individual
Legend
Static objects
Instances
Data